

Bison/Yacc

Thierson Couto Rosa

Instituto de Informática
Universidade Federal de Goiás

27 de fevereiro de 2026

Escopo da Aula

- Esta aula tem como objetivo dar informações suficientes para a implementação do analisador sintático do trabalho T1.
- Não é objetivo da aula mostrar todos os recursos da ferramenta Bison.

O que são o Yacc e o Bison

- O Yacc - *Yet another compiler compiler* é um gerador de analisador sintático - gerador de *parser* proposto por Stephen C. Johnson no início da década de 1970 e se tornou um utilitário do sistema Unix.
- O Bison é uma versão do Yacc com mais recursos desenvolvido pela GNU.
- Nestes slides serão apresentados os recursos do Bison, porém, o formato do arquivo de entrada para ambos é semelhante.
- O Bison recebe um arquivo de entrada contendo uma descrição de uma gramática livre de contexto do tipo LALR ou LR (serão estudadas com profundidade na segunda etapa da disciplina) e gera como saída um programa na linguagem C capaz de reconhecer se uma cadeia de entrada corresponde a uma cadeia gerada pela gramática.

- O arquivo de entrada para o Bison é uma mistura de texto contendo comandos e diretivas para o Bison e códigos na linguagem C - porque o Bison gera o parser em C.
- A entrada é organizada como segue:

```
%{
```

```
Declarações em linguagem C
```

```
%}
```

```
Definições
```

```
%%
```

```
Produções da gramática
```

```
%%
```

```
Funções do usuário escritas na linguagem C
```

- Os códigos em C na primeira e última seções são copiados no arquivo de saída gerado.

- A seção de definições (segunda seção), é onde se coloca configurações do *parser* como o código de definição dos tokens, definição de precedência e associatividade e iniciação de variáveis globais utilizadas para comunicação dos módulos em C do *parser* e analisador léxico.
- Na seção de produções da gramática, cada produção inicia com um símbolo não-terminal. A seta $\rightarrow$ é substituída por ' : '. Cada produção termina com um ' ; '. Alternativas de regras podem ser separadas por ' | '.

Entrada para o Bison correspondente a uma gramática de exemplo

- Considere a gramática: $G = (V_N = \{expression, term, factor\}, V_T = \{ID, NUM, +, -, /, (,)\}, P, expression)$, onde $P = \{$
 $expression \rightarrow expression + term \mid expression - term \mid term$
 $term \rightarrow term * factor \mid factor$
 $factor \rightarrow (expression) \mid ID \mid NUM$
 $\}$
- A adaptação deste exemplo como entrada para o Bison é mostrada no próximo slide.

Exemplo

```
1  %{
2  #include <stdio.h>
3  %}
4  %token ID
5  %token NUM
6  %%
7  expression : expression '+' term
8              | expression '-' term
9              | term
10             ;
11 term       : term '*' factor
12            | factor
13            ;
14 factor     : '(' expression ')'
15            | ID
16            | NUM
17            ;
18 %%
19 int main(){
20     return yyparse();
21 }
```

Algumas explicações

- O Bison espera que o analisador léxico retorne um código inteiro positivo correspondente a cada token de V_T . Quando um token é um caractere, como no caso de ' + ', ' - ', esse caractere pode ser utilizado diretamente na regra, pois o seu código na tabela ASCII é retornado pelo analisador léxico. Quando um token corresponde a uma linguagem regular não unitária, como ID e NUM, o definimos como %token ID e %token NUM. O analisador léxico utilizará um número inteiro superior a 256 para cada token especificado por %token.
- Se a entrada para o Bison estiver correta, ele gera o parser na linguagem C. O parser corresponde à função yyparse() e pode ser chamada pela função main() criada pelo usuário. A função yyparse() retorna 0 se o parser conseguiu analisar um exemplo de entrada e 1, em caso contrário.

Mais explicações

- Se `yyparse()` retorna 1, se o programador definir uma função `yyerror()`, ela será chamada. Essa função é útil para mostrar ao usuário que a cadeia de entrada não é válida na linguagem. No caso do exemplo anterior, a cadeia `x+y**k` não pertence à linguagem e `yyparser()` retorna 1 ao processá-la.
- A função `yyparser()` chama com recorrência o analisador léxico, o qual precisa retornar o código inteiro correto para cada token de V_T no exemplo acima.
- Na linha de comando, é possível informar ao Bison que gere um *header file* contendo as definições dos códigos dos tokens para que o analisador léxico possa saber quais valores inteiros dos tokens precisam ser retornados:
`bison --headerfile=tokens.h ex1.y -o parser.c` gera o arquivo `tokens.h` como *header file* e o arquivo e o *parser* no arquivo `parser.c`.

Exemplo - Gramática para a linguagem SimpleLang

- A gramática da SimpleLang é mostrada abaixo.

```
program → block
block → '{' stmts '}'
stmts → stmt stmts | ε
stmt → expr ';'
      | IF '(' expr ')' stmt
      | WHILE '(' expr ')' stmt
      | DO stmt WHILE '(' expr ')'
      | block
expr → rel '=' expr
      | rel
rel → rel '<' add
      | rel LEQ add
      | add
add → add '+' term | term
term → term '*' factor | factor ;
factor → '(' expr ')' | NUM
```

Entrada no Bison para a gramática de SimpleLang

```
1  %{
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <strings.h>
5  #include "tipos.h"
6  extern char * yytext;
7  extern int yylex();
8  extern FILE* yyin;
9  extern int erroOrigem;
10 void yyerror( char const *s);
11 %}
12 /* Definicao dos tokens que nao contem apenas um caractere*/
13 %token IF WHILE DO LEQ  NUM
```

```

1  %%  /* Secao de regras */
2
3  program :      block
4          ;
5
6  block   :      '{' stmts '}'
7          ;
8
9  stmts   :      stmt stmts
10         | /*cadeia vazia */
11         ;
12
13  stmt    :expr ';'
14         |IF '(' expr ')' stmt
15         |WHILE '(' expr ')' stmt
16         |DO stmt WHILE '(' expr ')'
17         |block
18         ;
19
20  expr    :rel '=' expr
21         | rel
22         ;

```

```
1 rel      : rel '<' add
2           | rel LEQ add
3           | add
4           ;
5 add       : add '+' term
6           | term
7           ;
8 term: term '*' factor
9           | factor
10          ;
11 factor   : '(' expr ')'
12           | NUM
13          ;
```

```

1  %% /* Secao Epilogo*/
2  int main(int argc, char** argv){
3      if(argc!=2)
4          printf("Uso correto: ./simpleLang nome_arq_entrada");
5      yyin=fopen(argv[1], "r");
6      if(!yyin)
7          printf("arquivo nao pode ser aberto\n");
8      yyparse();
9  }
10
11 void yyerror( char const *s) {
12     if(erroOrigem==0) /*Erro lexico*/
13     {
14         printf("%s na linha %d - token: %s\n", s, numLinha,
15             yytext);
16     }
17     else
18     {
19         printf("Erro sintatico proximo a %s", yytext);
20         printf("- linha: %d\n", numLinha);
21         erroOrigem=1;
22     }
23     exit(1);
24 }

```

- O Bison gera um *parser* (`yyparse()`) para uma gramática dada com entrada. Porém ele precisa de uma função que obtenha o próximo token no arquivo de entrada. O parser `yyparse()` chama uma função `yylex()` para essa finalidade. Essa função não é gerada pelo Bison ou Yacc. Precisa ser gerada pelo programador ou por uma outra ferramenta geradora de analisador léxico, o Lex ou Flex.
- Apresentaremos o Flex no próximo conjunto de slides, mas você já deve gerar o parser para a linguagem especificada no trabalho T1.